Sisteme de operare
Laborator 7
POSIX threads

1. Creati un nou subdirector *lab7/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu PATH pentru a include directorul *SO/laborator/lab7/bin*.

2. Scrieti un program C **shm.c** care creeaza o zona de memorie partajata intre un proces parinte si un copil. Parintele creeaza zona de memorie partajata (puteti folosi IPC_PRIVATE) si o ataseaza la proces inainte de apelul *fork*. Procesul copil executa un loop cu 10 iteratii in care scrie succesiv, la fiecare iteratie, urmatorul mesaj in zona de memorie partajata:

"this is the child printing " + numarul iteratiei

Pentru a compune asemenea mesaje puteti folosi functia de biblioteca *sprintf*. Mesajele din fiecare iteratie se adauga (*append*) la sfarsitul mesajului scris in iteratia anterioara (cu exceptia primei iteratii).

Procesul parinte executa si el un loop cu 10 iteratii in care, la fiecare iteratie, citeste unul dintre mesajele scrise de procesul copil si le afiseaza pe ecran.

Dezvoltati o solutie folosind semafoare System V sau semafoare POSIX cu nume care sa permita coordonarea (sincronizarea) accesului celor doua procese la zona de memorie partajata a.i. executia programului sa fie cea anticipata, si anume dupa fiecare mesaj scris de copil in memoria partajata parintele il citeste si il scrie pe ecran. (*Obs*: *ambele procese executa un loop cu 10 iteratii dar nu puteti face nici o presupunere asupra vitezei relative de executie a celor doua procese*).

La final, apelati *shmctl* si *semctl* cu IPC_RMID pentru a sterge memoria partajata si semafoarele din sistem (eventual *shm_unlink*, daca folositi semafoare POSIX cu nume).

3. Modificati programul C **msg-echod.c** din laboratoarele trecute pentru a oferi un serviciu ***multi-threaded*** de *echo* folosind cozi de mesaje System V IPC. Noul program, **msg-mt-echod.c**, creeaza o coada de mesaje cu ajutorul *msgget* (parametrul *key* il generati cu functia *ftok*). Apoi apeleaza in bucla functia *msgrcv* pentru a primi mesaje de la clienti, Spre deosebire de **msg-echod.c**, care este *single-threaded*, **msg-mt-echod.c** creeaza un thread POSIX separat pentru fiecare nou mesaj primit. Serverul copiaza mesajul primit cu *msgrcv* si il paseaza noului thread creat care il scrie inapoi in coada de mesaje (serviciu de *echo*) folosind *msgsnd*. Programul **msg-mt-echod.c** se lanseaza in background ca serviciu de *echo* dupa cum urmeaza:

```
$ gcc -o echod  msg-mt-echod.c
$ echod keyfile &                  # keyfile e un fisier pe care il creati voi
$ ipcs -q                          # vizualizati coada creata
```

Pentru a testa functionalitatea serverului folositi programul client **msg-echo.c** dezvoltat in laboratoarele trecute.

La final, nu uitati sa faceti curatenie:

```
$ jobs
```

```
$ kill %<n>                 # n e numarul de job al echod afisat de comanda
                            # jobs de mai sus
$ ipcs -q
$ ipcrm -q <id>             # id e id-ul cozii de mesaje tiparit de comanda
                            # ipcs de mai sus
```

4. Rescrieti programul C **race.c** din laboratorul 6 in care in loc sa folositi doua procese, parinte si copil, care scriu concurent, caracter cu caracter, propriul string pe ecran, veti folosi doua thread-uri POSIX. Pentru a evidentia mai clar race condition-ul, fiecare thread repeta operatia de tiparire pe ecran de un numar de ori, in bucla. Folositi un semafor POSIX pentru a impiedica producerea race condition-ului si a obtine un output neintretesut pe ecran. La final, nu uitati sa stergeti semaforul din sistem.

Obs: Avand in vedere ca programul foloseste thread-uri, puteti folosi semafoare POSIX fara nume direct, apeland *sem_init* cu parametrul corespunzator.

5. Scrieti un program C **prod-cons.c** care implementeaza o solutie a problemei producator-consumator folosind doua thread-uri POSIX, unul pentru producator si unul pentru consumator. Pentru implementarea sincronizarii folositi monitoare (mai exact, fiind vorba de POSIX, un mutex si doua variabile conditie, una pentru buffer plin si alta pentru buffer gol).